

Reproducing Robust Cross-Image Adversarial Watermark

Detailed Implementation Guide & Technical Specification | Lin et al. (2025)

1. Executive Summary & Architectural Overview

This document outlines the end-to-end technical strategy for reproducing the proactive Deepfake defense framework presented by Lin et al. (2025). The target system generates a single, cross-image universal adversarial watermark embedded directly into the **Discrete Cosine Transform (DCT) frequency domain**. Unlike standard spatial perturbations that are wiped out by JPEG lossy compression, this frequency-domain watermark survives transmission and quantization, forcing targeted GAN-based Deepfake generators (such as StarGAN, AttGAN, and HiSD) to produce severely distorted facial outputs.

Core Defense Mechanism

Given an input face x and target attribute c , a Deepfake model computes $x' = G(x, c)$. By optimizing a frequency watermark W embedded into DCT luminance coefficients, the protected output $G(x + W, c)$ fails to synthesize the target attribute, yielding severe visual artifacts while keeping the unedited protected image $x + W$ visually imperceptible from x .

2. Complete Module Breakdown & Equations

2.1 Color Space & Discrete Cosine Transform (DCT)

Images are first converted from RGB to the YCbCr color space. Watermark optimization is restricted exclusively to the **Y (luminance) channel**, while Cb and Cr channels remain untouched to preserve color fidelity:

$$y = E_n(I) \quad \text{— where } E_n \text{ performs RGB} \rightarrow \text{YCbCr, } 8 \times 8 \text{ block splitting, and DCT transformation.}$$

2.2 Differentiable JPEG Compression Module

Standard JPEG quantization utilizes a discontinuous rounding operation $\text{round}(x)$, causing zero gradients almost everywhere. To allow end-to-end backpropagation via Projected Gradient Descent (PGD), standard rounding is replaced with a smooth cubic approximation:

$$x_{\text{approx}} = \text{round}(x) + (x - \text{round}(x))^3$$

The differentiable JPEG decoder $D_e(y + W)$ takes the watermarked DCT coefficients, applies differentiable inverse quantization, inverse DCT, and YCbCr $\rightarrow$ RGB spatial conversion to yield the protected spatial image I^{inv} .

2.3 Optimization Objective & Loss Function

The adversarial watermark W is optimized using a compound loss function balancing Deepfake output disruption against visual imperceptibility:

$$L = L_{\text{MSE}}(G(I^{\text{inv}}), G(I)) + \lambda \cdot (1 / L_D)$$

Where L_{MSE} calculates the Mean Squared Error between the Deepfake output of the protected image and clean image, $L_D = PSNR(I^{inv}, I)$ evaluates spatial image fidelity, and $\lambda = 0.05$ balances attack aggressiveness with perceptual stealthiness.

2.4 Cross-Image Fusion Strategy

To ensure transferability across arbitrary faces without retraining:

- 1. **Intra-Batch Gradient Averaging:** Within a training batch of size $n = 8$, gradients are averaged across samples to extract common facial features:

$$grad = (1 / n) \sum sign(\nabla_{I_i} L)$$

- 2. **Inter-Batch Exponential Fusion:** Across sequential batches m , the universal watermark is updated dynamically using exponential smoothing ($\beta = 0.50$):

$$W^{m+1} = \beta \cdot W^m + (1 - \beta) \cdot w^m$$

3. Exact Experimental Setup & Hyperparameters

Parameter / Setting	Target: StarGAN	Target: AttGAN	Target: HiSD
Perturbation Budget (ϵ)	0.20	0.18	0.30
PGD Iterations (T)	15	15	20
Step Size (α)	0.03	0.03	0.03
Dataset Setup	128 training images (16 batches $\times$ 8 images), 1 Epoch training		
Training Quality Factor (q)	q = 35 (Balancing PSNR > 30 dB with high defense performance)		
Evaluation Standard	JPEG Q = 75 (Default social media compression setting)		

4. Step-by-Step Reproduction Blueprint

Step 1: Environment Setup & Pretrained Models

- Setup PyTorch environment with CUDA support.
- Download evaluation datasets: CelebA test split and LFW. Resize images to 256 $\times$ 256.
- Load official pretrained weights for **StarGAN**, **AttGAN**, and **HiSD** and freeze generator weights.

Step 2: Custom Differentiable JPEG Module Construction

- Implement PyTorch layer for RGB $\leftrightarrow$ YCbCr conversion using fixed color matrices.
- Implement 2D Discrete Cosine Transform (DCT) using 8 $\times$ 8 block convolutions or standard matrix multiplications.
- Implement differentiable quantization layer using $x_{approx} = round(x) + (x - round(x))^3$ with standard JPEG luminance quantization matrices scaled by quality factor $q = 35$.

Step 3: PGD Optimization & Fusion Implementation

- Initialize random noise tensor W_{init} in DCT space.
- Iterate through 16 batches of 8 clean images. For each batch, run T steps of PGD in frequency space.
- Compute batch average gradient and update local batch watermark w^m .
- Apply inter-batch fusion: $W^{m+1} = 0.5 \cdot W^m + 0.5 \cdot w^m$.

Step 4: Evaluation & Metrics Calculation

- Save final universal watermark $W = W_{17}$.
- Apply W to 1,000 unseen test faces. Compress protected images using standard non-differentiable JPEG ($Q = 75$).
- Feed compressed protected images into Deepfake generators and calculate SR_{mask} (using artifact-aware threshold $L_2^{mask} > 0.05$), PSNR, SSIM, and MSE.

5. Ready-to-Use LLM Implementation Prompt

Copy and paste the prompt below into any coding LLM (e.g., Claude 3.5 Sonnet, GPT-4o) to automatically generate the complete PyTorch implementation.

SYSTEM PROMPT / TASK ASSIGNMENT:

You are an expert Computer Vision and PyTorch AI Engineer. Implement the complete, fully functional Python codebase to reproduce the paper "Robust cross-image adversarial watermark with JPEG resistance for defending against Deepfake models" (Lin et al., CVIU 2025).

REQUIREMENTS & MODULE SPECIFICATIONS:

1. Differentiable JPEG Module (DiffJPEG):

- Build a custom PyTorch module with `encode(x)` and `decode(y)` functions.
- Convert RGB images (256x256, [0, 1]) to YCbCr color space.
- Perform 8x8 block Discrete Cosine Transform (DCT) on the Y (luminance) channel.
- Implement differentiable quantization using smooth approximation:
$$x_{approx} = \text{torch.round}(x) + (x - \text{torch.round}(x)) ** 3$$
- Use standard JPEG luminance quantization matrix scaled by quality factor $q = 35$.

2. Adversarial Watermark Optimization & Fusion Engine:

- Implement Projected Gradient Descent (PGD) in DCT frequency space.
- Objective Loss: $L = \text{MSE}(G(I_{inv}), G(I)) + \text{lambda_val} * (1.0 / \text{PSNR}(I_{inv}, I))$ with $\text{lambda_val} = 0.05$.
- Intra-batch Gradient Averaging: $\text{grad_avg} = \text{torch.mean}(\text{torch.sign}(\text{grad}), \text{dim}=0)$.
- Inter-batch Exponential Fusion: $W_{\{m+1\}} = \text{beta} * W_m + (1 - \text{beta}) * w_m$ with $\text{beta} = 0.50$.
- Training Setup: 128 images, 16 batches of 8 images, 1 epoch.
- Parameters for StarGAN: $\text{epsilon} = 0.20$, $T = 15$, $\text{alpha} = 0.03$.

3. Evaluation Pipeline:

- Add utility to apply final universal watermark W to unseen test images.
- Simulate real non-differentiable JPEG compression ($Q = 75$) using PIL/io BytesIO.
- Pass compressed images into pretrained Deepfake model G .
- Compute metrics: PSNR, SSIM, MSE, L_2_{mask} , and Defense Success Rate ($SR_{mask} \text{ threshold} > 0.05$).

Write modular, clean, PEP-8 compliant PyTorch code with full docstrings, assertions, and a runnable `main()` routine simulating the workflow on synthetic tensors.